

Operators in Python

Operators are symbols used to perform operations on values and variables. They allow you to calculate, compare, assign, and combine data in different ways.

Python provides several types of operators, each serving a specific purpose.

Arithmetic Operators

Arithmetic operators are used to perform mathematical calculations.

Operator	Description	Example
+	Addition	10 + 5
-	Subtraction	10 - 5
*	Multiplication	10 * 5
/	Division	10 / 5
//	Floor Division	10 // 3
%	Modulus (remainder)	10 % 3
**	Exponent	2 ** 3

Example:

a = 10

b = 3

print(a + b)

print(a // b)

print(a ** b)

Assignment Operators

Assignment operators are used to assign values to variables.

Operator	Description	Example
=	Assign	x = 5
+=	Add and assign	x += 2
-=	Subtract and assign	x -= 2
*=	Multiply and assign	x *= 2
/=	Divide and assign	x /= 2
%=	Modulus and assign	x %= 2
//=	Floor divide and assign	x //= 2
**=	Exponent and assign	x **= 2

Example:

x = 10

x += 5

Comparison Operators

Comparison operators compare two values and return either True or False.

Operator	Description	Example
==	Equal to	a == b
!=	Not equal to	a != b
>	Greater than	a > b
<	Less than	a < b

Operator	Description	Example
>=	Greater than or equal	a >= b
<=	Less than or equal	a <= b

Example:

a = 10

b = 5

print(a > b)

Logical Operators

Logical operators are used to combine conditional statements.

Operator	Description	Example
and	Both true	a > 5 and b < 10
or	Either true	a > 5 or b > 10
not	Reverse	not(a > 5)

Example:

a = 10

b = 3

print(a > 5 and b < 5)

Bitwise Operators

Bitwise operators work on binary representations of numbers.

Operator	Description	
&	AND	
	OR	

Operator	Description	
<code>^</code>	XOR	
<code>~</code>	NOT	
<code><<</code>	Left shift	
<code>>></code>	Right shift	

Example:

```
a = 5
```

```
b = 3
```

```
print(a & b)
```

Membership Operators

Membership operators are used to test whether a value exists in a sequence.

Operator	Description
<code>in</code>	Exists in sequence
<code>not in</code>	Does not exist

Example:

```
numbers = [1, 2, 3]
```

```
print(2 in numbers)
```

Identity Operators

Identity operators compare memory locations of objects, not values.

Operator	Description
is	Same object
is not	Different object

Example:

```
a = [1, 2]
```

```
b = a
```

```
print(a is b)
```

Operator Precedence

Python evaluates operators in a specific order. Operators with higher precedence are evaluated first.

Precedence Order	Operators	Description
1	()	Parentheses
2	**	Exponentiation
3	* / // %	Multiplication / Division
4	+ -	Addition / Subtraction
5	== != > <	Comparison
6	not and or	Logical operators

Operators at the top are evaluated before operators lower in the table.

Using parentheses is the safest way to control evaluation order and make expressions easier to read.